Model Context Protocol (MCP)

Professor Peerapon Vateekul, Ph.D.
peerapon.v@chula.ac.th

Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University

Outline

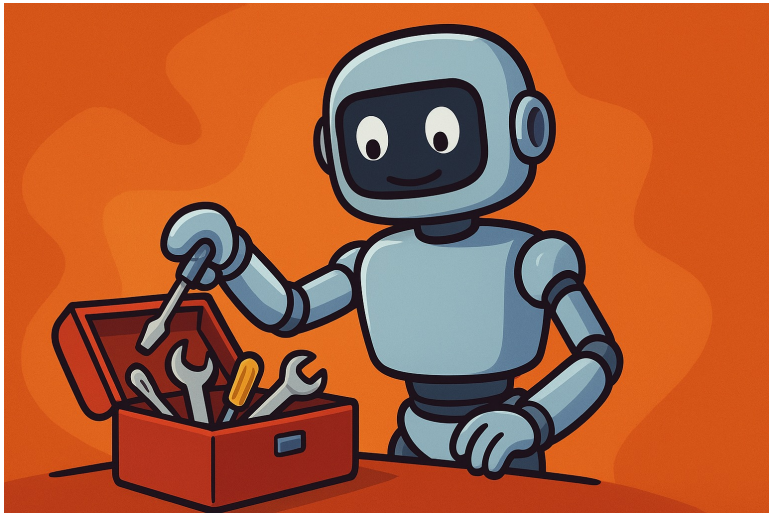
- Learning objectives
- Model Context Protocol (MCP)
- Why LLMs need external tools? External RAG/Tools
- MCP connects AI to tools and data
- REST API vs MCP
- The Three Roles: Host, client, and server
- MCP use cases
- Lab 1: Tomorrow.io MCP
- Lab 2: Our FastMCP Server (Weather Data)

Learning objectives

1) Explain MCP and compare it with a REST API

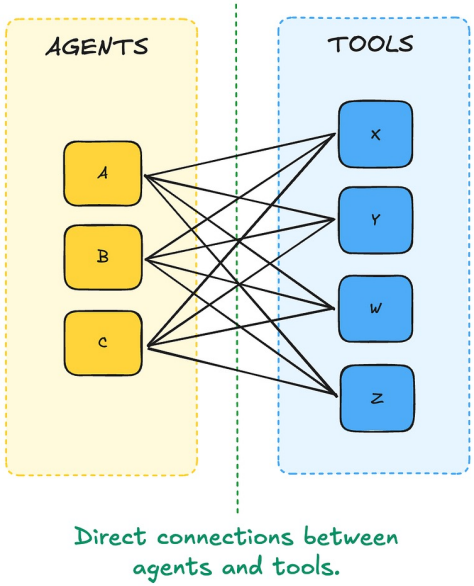
2) Use a remote weather service through MCP (Tomorrow.io)

3) Expose Pandas functions with FastMCP

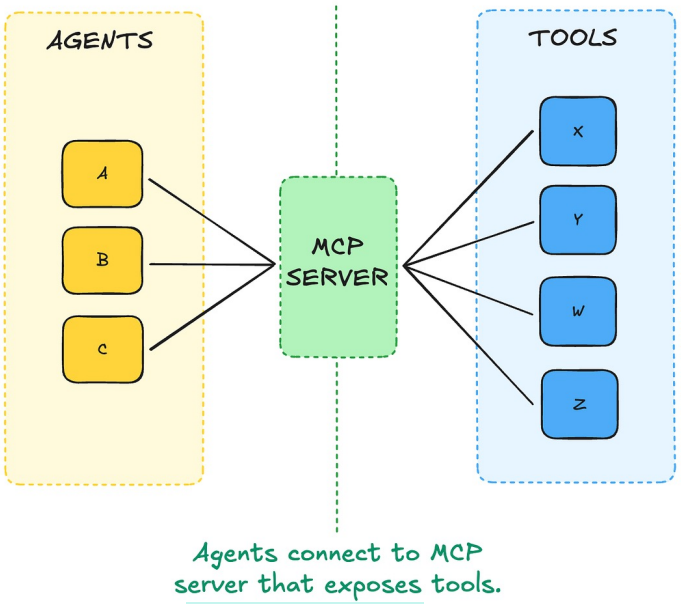


Model Context Protocol (MCP)

- MCP is an open standard introduced by Anthropic in late 2024.
- Enables AI applications to discover, authenticate with, and call external tools.
- Acts as a universal adapter connecting LLMs to external software and data.



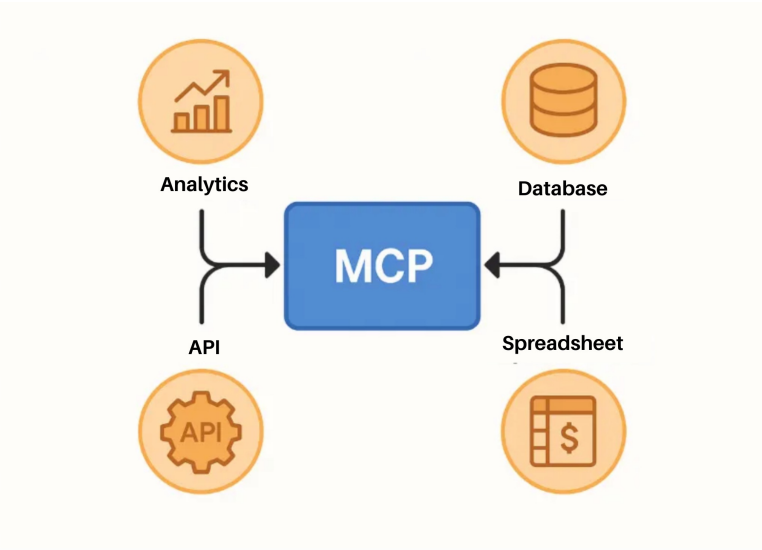
Without MCP



With MCP

Why LLMs need external tools? External RAG/Tools

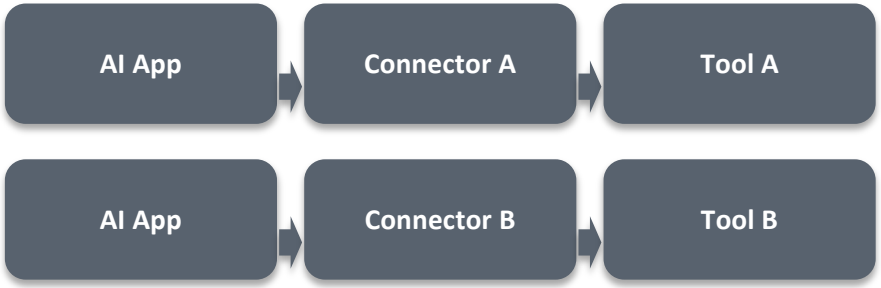
Question	Required external evidence
What is the weather in Bangkok now?	Current conditions for a known location
Which province has the highest average rainfall?	The actual dataset and aggregation logic
What files are in my project folder?	Access to the permitted directory



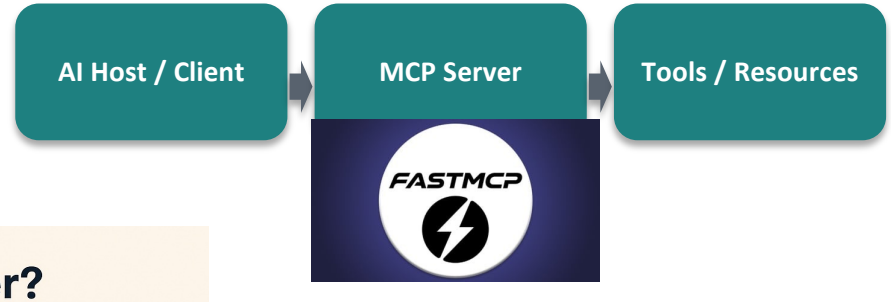
MCP connects AI to tools and data

MCP standardizes how AI applications connect to external tools and context sources.

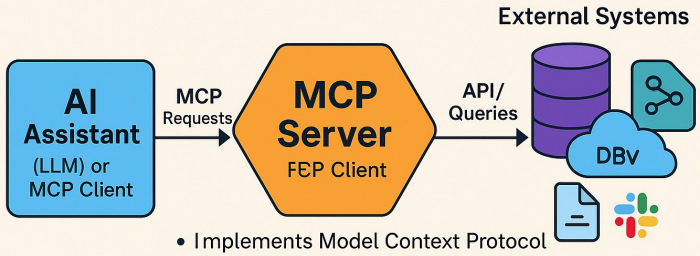
Before: custom integrations



With MCP: standardized interface



What is an MCP Server?



- Implements Model Context Protocol
- Translates AI intents into safe, granular operations
- Enforces permissions
- Returns structured context back to model

MCP Server bridges AI ↔ Data/Tools

REST API vs MCP

REST API

- **REST API: Developers specify the endpoint and request format.**
- **Known endpoints and HTTP methods**
- **Service-specific operations**
- **OpenAPI can describe schemas**

MCP

- **MCP: The AI discovers available tools and selects a suitable weather tool.**
- **Standard capability discovery**
- **Tool schemas and calls**
- **Resources and prompt templates**

Example: “What is the weather in Bangkok?”

REST: Call an endpoint.

```http
GET https://api.tomorrow.io/v4/weather/realtime
    ?location=Bangkok
    &units=metric
    &apikey=YOUR_API_KEY
```

MCP: Discover and call a named tool.

```python
tools = await client.list_tools()

result = await client.call_tool(
    "get_top_provinces",
    {"column": "rainfall", "n": 1}
)
```

The Three Roles: Host, client, and server

Role	Example in this lab
Host / application	The application you use: Claude Desktop, Cursor, VS Code with Copilot, Claude Code.
MCP client	A connector inside the host
MCP server	The program you connect to Example: Tomorrow.io or mcp_server.py

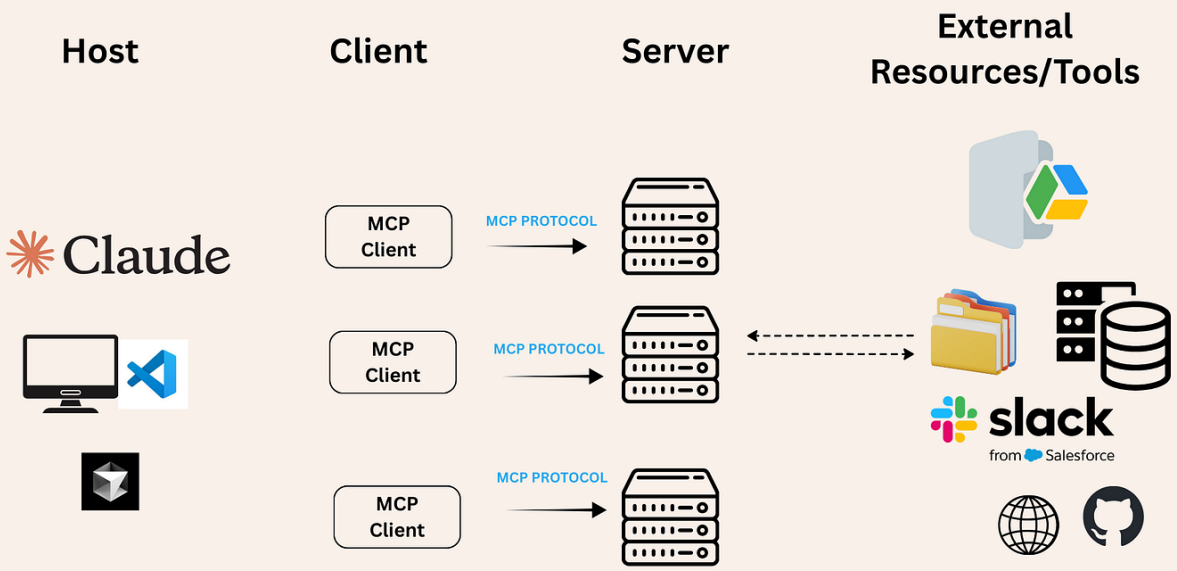


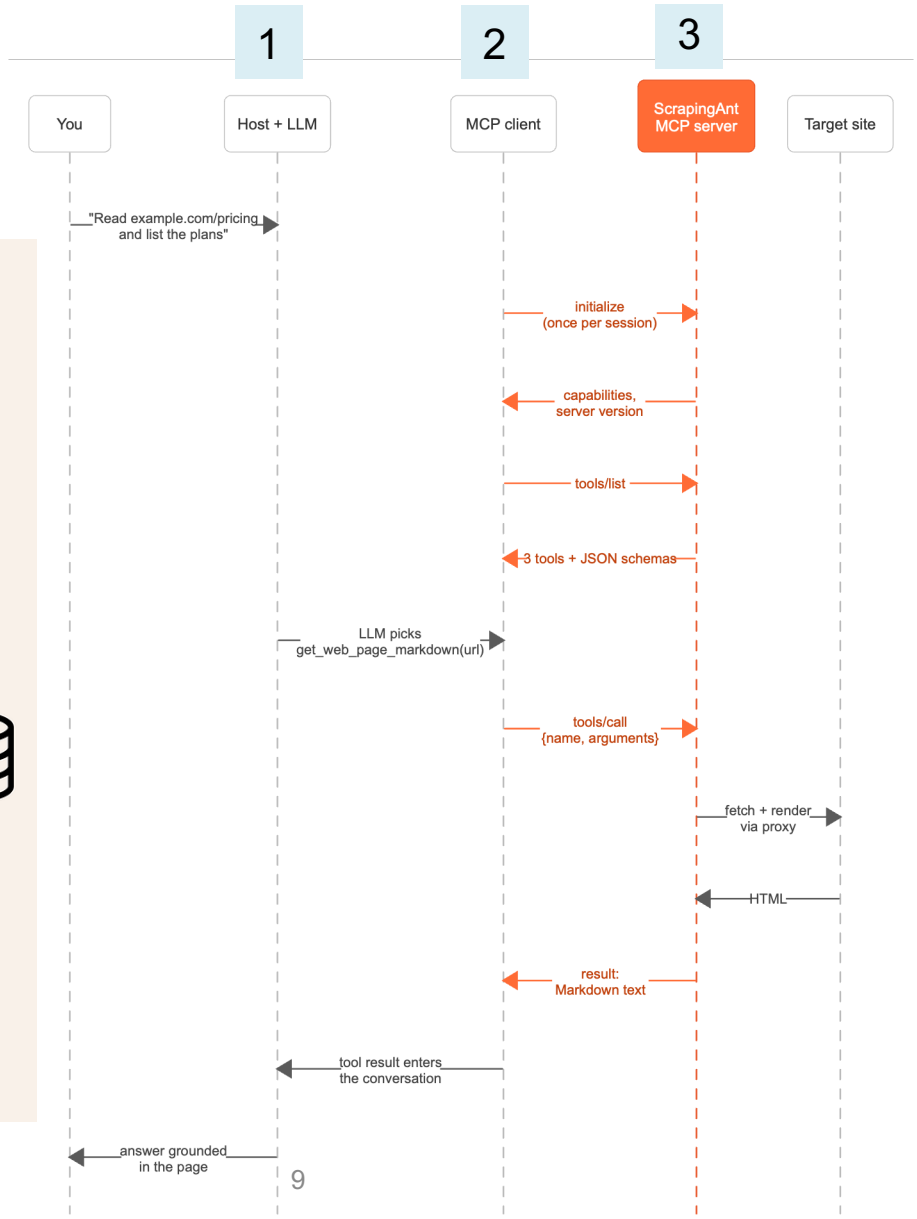
MCP Server can provide:

Primitive	Purpose	Example
Tool	Execute a capability	Rank provinces
Resource	Read contextual information	Dataset schema
Prompt	Reuse an interaction template	Guided EDA steps

LLM discovers & calls tools automatically

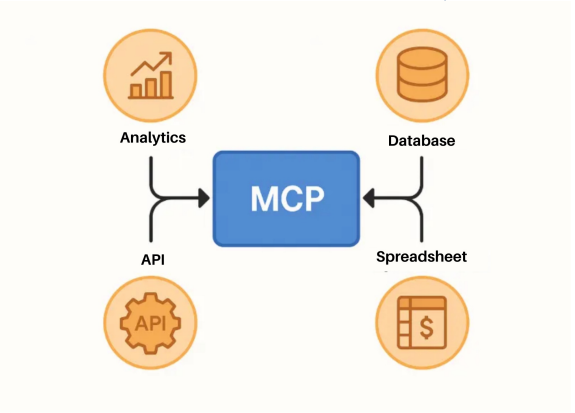
MCP Architecture





MCP use cases

Use case	Example request
Weather	What is the weather in Bangkok now?
Data analysis	Which province has the highest average rainfall?
Local files	List the CSV files in my project folder.
GitHub	Summarize open issues in our repository.
Browser interaction	Check whether our dashboard filter works.



References and lab sources in speaker notes

Use case: a weather question

What is the weather in Bangkok now?

1. The user names a location and time
2. The assistant selects a weather tool
3. Tomorrow.io returns current conditions
4. The assistant reports the result with its timestamp

Use case: a dataset question

Which province has the highest average rainfall?

```
get_top_provinces(column="rainfall", n=1)

[{"province": "Phuket", "average": 2300.0}]
```

Pandas calculates the result. The LLM explains it.

Lab 1: Tomorrow.io MCP

The Tomorrow.io endpoint in our notebook

https://api.tomorrow.io/v4/weather-intelligence/mcp

Remote MCP App over Streamable HTTP

No Tomorrow.io API key for this endpoint

Weather, air quality, pollen, and fire-risk capabilities

Remote MCP request in the notebook

```
tools=[
    {
        "type": "mcp",
        "server_label": "weather_intelligence",
        "server_url": (
            "https://api.tomorrow.io/v4/weather-intelligence/mcp"
        ),
        "require_approval": "never"
    }
]
```

Weather output: what to inspect

Field / property	Review question
Location	Did the tool resolve Bangkok correctly?
Time	When does the observation or forecast apply?
Units	Are values metric or imperial?
Data type	Current conditions or a forecast?
Answer support	Does the tool output support the final text?

Lab 2: Our FastMCP Server (Weather Data)



FastMCP: Python functions as tools

```
from fastmcp import FastMCP

mcp = FastMCP("Data Science Assistant")

@mcp.tool
def get_summary() -> dict:
    """Return dataset shape, columns, and statistics."""
    ...
```

Type hints describe inputs and outputs

Docstrings explain the tool’s purpose

The dataset in mcp_server.py

Province	Year	Rainfall	Crop yield
Chiang Mai	2023	1100	3.1
Chiang Mai	2024	1250	3.4
Bangkok	2023	1400	2.8
Bangkok	2024	1550	3.0
Phuket	2023	2400	2.5
Phuket	2024	2200	2.7

Weather data in memory. Units are unspecified.

Three tools exposed by the server

Tool	Input	Purpose
get_summary	None	Describe the dataset
get_top_provinces	column, n=5	Rank provincial means
filter_data	province	Return matching records

Tool 1: dataset summary

```
@mcp.tool
def get_summary() -> dict:
    return {
        "rows": len(df),
        "columns": df.columns.tolist(),
        "statistics": df.describe().round(2).to_dict(),
    }
```

A summary establishes what data is available

Tool 2: validation before ranking

```
@mcp.tool
def get_top_provinces(column: str, n: int = 5):
    allowed_columns = {"rainfall", "crop_yield"}

    if column not in allowed_columns:
        raise ValueError("Unsupported column")

    if not 1 <= n <= 20:
        raise ValueError("n must be between 1 and 20")
```

An allowlist limits which metrics the tool accepts

Tool 2: aggregation and ranking

```
return (
    df.groupby("province")[column]
    .mean()
    .sort_values(ascending=False)
    .head(n)
    .round(2)
    .rename("average")
    .reset_index()
    .to_dict(orient="records")
)
```

Tool 3: filtering province records

```
@mcp.tool
def filter_data(province: str) -> list[dict]:
    result = df[
        df["province"].str.lower() == province.lower()
    ]
    return result.to_dict(orient="records")
```

Input	Expected behaviour
"phuket"	Returns two Phuket records
"Unknown"	Returns an empty list

MCP Client

- MCP Client 1: Testing without an LLM (mcp_client.py → mcp_server.py)
- MCP Client 2: The LLM tool-calling loop

MCP Client 1: Testing without an LLM (mcp_client.py → mcp_server.py)

```
from fastmcp import Client
from mcp_server import mcp

async with Client(mcp) as client:
    result = await client.call_tool(
        "get_top_provinces",
        {"column": "rainfall", "n": 3}
    )

print(result)
```

Client(mcp) uses an in-memory connection

MCP Client 2: The LLM tool-calling loop

1. Discover tools with client.list_tools()
2. Pass tool schemas to the model
3. Read the model's tool name and arguments
4. Execute client.call_tool(name, args)
5. Send the tool result back to the model
6. Return an answer or continue the loop

```python
async def ask_data_local_import_mcp(question: str):
    messages = get_prompt(question)
    # Local MCP client: tool execution happens in this notebook.
    async with Client(mcp) as client:
        mcp_tools = await client.list_tools()

        tools = [
            {
                "type": "function",
                "function": {
                    "name": tool.name,
                    "description": tool.description or tool.name,
                    "parameters": tool.input_schema,
                },
            }
            for tool in mcp_tools
        ]

        allowed_names = {tool.name for tool in mcp_tools}

        ###
        # Loop notes
        # 1. [sys, user ] -> ask = msg
        # 2. [sys, user, msg ] -> call tool = tool_output
        # 3. [sys, user, msg, tool_output ] -> answer !
        ###

        for _ in range(5):  # Limit tool-calling rounds
            response = await groq.chat.completions.create(
                model=MODEL,
                messages=messages,
                tools=tools,
                tool_choice="auto",
            )

            msg = response.choices[0].message
```

References

- https://medium.com/@ale.garavaglia/building-an-mcp-server-with-fastmcp-a-hands-on-demo-98208c5370df
- https://scrapingant.com/blog/what-is-mcp-server
- https://2slides.com/th/blog/how-mcp-model-context-protocol-changing-presentation-workflows